

Clase 131 — Ghidra para ingeniería inversa

Parte: **5 — Explotación de sistemas y binarios** · Fuente: *Andriesse, Practical Binary Analysis* · docs de la NSA/Ghidra 🕒 Duración estimada: **130 min** · Nivel: **Intermedio**

🎯 Objetivo

Aprender a usar **Ghidra**, el framework libre de ingeniería inversa de la NSA, para desensamblar y **decompilar** binarios a pseudo-C legible. Verás cómo crear un proyecto, navegar por funciones, renombrar variables, corregir tipos y usar el decompilador para entender la lógica de un `crackme` mucho más rápido que leyendo ensamblador puro.

⚠️ **Ética:** analiza solo binarios propios/autorizados.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Crear** un proyecto Ghidra e importar/analizar un binario.
- 2. **Navegar** entre listing (ASM) y decompiler (pseudo-C).
- 3. **Renombrar** funciones/variables y **retipar** datos para mejorar la lectura.
- 4. **Seguir** referencias cruzadas (xrefs) hacia y desde una función.
- 5. **Automatizar** tareas con scripts (Python/Jython).

📖 Temas

#	Tema	Por qué importa
1	Proyecto e importación	Punto de partida
2	Auto-análisis	Ghidra reconstruye funciones y tipos
3	Listing vs Decompiler	ASM detallado vs C legible
4	Renombrar y comentar	Documentar hallazgos
5	Retipado de estructuras	Legibilidad de structs/arrays
6	Xrefs	Rastrear flujo de datos y llamadas
7	Bookmarks y symbol tree	Organizar el análisis
8	Scripting (GhidraScript)	Automatizar y extraer

Definiciones y características

- **Ghidra**: SRE framework de código abierto con decompilador propio. *Clave*: gratuito y multiplataforma; el decompilador rivaliza con IDA.
- **Decompilador**: genera pseudo-C a partir del ensamblado. *Clave*: mejora enormemente al renombrar y retipar variables.
- **Listing**: vista de desensamblado con direcciones, bytes y comentarios. *Clave*: fuente de verdad cuando el decompilador se equivoca.
- **Xref (cross-reference)**: enlace entre una dirección y quienes la referencian. *Clave*: `Ctrl+Shift+F` para ver quién llama a una función.
- **Symbol Tree / Data Type Manager**: paneles para funciones, imports/exports y tipos. *Clave*: definir structs mejora todo el decompilado.
- **GhidraScript**: API para automatizar (Python/Jython o Java). *Clave*: útil para desofuscar o extraer cadenas en lote.

Herramientas y preparación

```
# Requiere JDK; descarga Ghidra desde el sitio oficial:
# https://ghidra-sre.org/ (o el repo de GitHub de la NSA)
# Descomprime y ejecuta:
./ghidraRun
```

Ten a mano un `crackme` de práctica (por ejemplo, el de la clase 130).

Laboratorio guiado

Entorno propio.

1. Crea un proyecto no compartido (`File → New Project`), importa el `crackme` y acepta el **auto-análisis**.
2. En el Symbol Tree abre `main` ; observa el panel Decompiler (pseudo-C) junto al Listing.
3. Localiza la comparación de la clave. Renombra variables genéricas (`local_28` → `input` , `uVar1` → `len`) con `L` para clarificar la lógica.
4. Sigue las xrefs de la cadena del prompt: haz doble clic en la string en Defined Strings y usa `References → Show References to` .
5. Retipa un buffer como `char[32]` para que el decompilador muestre la clave esperada de forma legible.
6. Deduce la contraseña/serial válido a partir del pseudo-C (comparación, transformación, longitud).
7. (Opcional) Escribe un GhidraScript en Python que liste todas las funciones que llaman a `strcmp` .
8. Verifica tu hipótesis ejecutando el `crackme` con la clave deducida.

Ejercicios

1. Renombra y comenta `main` hasta que el pseudo-C sea autoexplicativo.
2. Define una `struct` para un objeto que el binario usa y aplícala.
3. Usa xrefs para encontrar todas las llamadas a la función de validación.
4. Extrae con un script la lista de imports del binario.
5. Compara el Listing y el Decompiler en una función donde difieran.
6. Exporta un informe/anotaciones del análisis.

Reto verificable

Usando solo Ghidra (sin ejecutar hasta el final), deduce la clave válida de un `crackme` y luego compruébala ejecutándolo.

Criterio de aceptación: el `crackme` acepta la clave que dedujiste del pseudo-C, y documentas en comentarios de Ghidra cómo llegaste a ella.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Decompilado ilegible	Renombra/retipa variables; define structs
Ghidra no arranca	Falta JDK compatible; instala el JDK requerido
Funciones no reconocidas	Reejecuta auto-análisis o define funciones manualmente
El decompilador "miente"	Contrasta con el Listing (ASM) real
Strings no aparecen	Están cifradas; combínalo con análisis dinámico

Preguntas frecuentes

 **¿Ghidra o IDA?** Ghidra es gratuito y muy capaz; IDA es el estándar comercial. Aprende ambos si puedes (IDA/radare2 en la clase 132).

 **¿El decompilado es fiable?** Es una aproximación; para detalles finos (offsets, flags) confía en el Listing.

 **¿Puedo automatizar?** Sí, con GhidraScript o el modo headless (`analyzeHeadless`) para lotes.

Referencias

- Ghidra (NSA) — <https://ghidra-sre.org/>
- Ghidra en GitHub — <https://github.com/NationalSecurityAgency/ghidra>
- Andriesse, D. *Practical Binary Analysis*. No Starch Press.
- The Ghidra Book (Eagle & Nance), No Starch Press.

Material descargable

-  **Guía en PDF** — versión imprimible de esta clase.
-  **Presentación (PPTX)** — deck para proyectar en clase.

Siguiente clase

Clase 132 - IDA Pro y radare2